

Document Number: P1422R0
Date: 2019-01-17
Audience: SG16
Reply-to: Tom Honermann <tom@honermann.net>

SG16: Unicode meeting summaries 2018/10/17 - 2019/01/09

Summaries of SG16 meetings are maintained at <https://github.com/sg16-unicode/sg16-meetings>. This paper contains a snapshot of select meeting summaries from that repository.

- [October 17th, 2018](#)
- [December 5th, 2018](#)
- [December 19th, 2018](#)
- [January 9th, 2019](#)

October 17th, 2018

Draft agenda:

- char8_t: Markus' concerns, motivation, type safety, Unicode sandwich, most C++ code is yet to be written, transition story.
- Code points, EGCs, or explicit ranges for text views/containers?
 - How to decide? Pick a direction now? Write a pros/cons paper for the committee?

Attendees:

- Artem Tokmakov
- Cameron Gunnin
- JeanHeyd Meneide
- Mark Zeren
- Markus Scherer
- Martinho Fernandes
- Sergey Zubkov
- Steve Downey
- Tom Honermann
- Zach Laine

Meeting summary:

- [Issue #30: Unclear behavior for octal and hex escape sequences in Unicode character and string literals](#)
 - Tom explained the current situation; [CWG#2333](#) tracks this issue. CWG discussed at their August 2017 teleconference and decided that numeric escape sequences should be ill-formed in UTF-8 character literals. Mike Miller offered to reconsider the issue if requested by SG16.
 - Markus mentioned the utility in using numeric escapes to create ill-formed strings for testing purposes.
 - Markus also presented an alternative possibility, that numeric escapes only be ill-formed if used to encode a code unit value that is never valid in a UTF string, e.g., `0xff`.
 - Markus additionally noted that there is a distinction between Unicode strings (may contain ill-formed contents) and UTF strings (must be well-formed).
 - Zach asserted that the ability to use numeric escapes is more important than preventing encoding of ill-formed UTF sequences.

- Tom noted that the current CWG resolution seems evolutionary given that it contradicts existing practice.
- Markus noted a further benefit, maintaining consistency with languages like Java. Additionally, he explained that some logging libraries write strings with non-printable characters replaced with escape sequences and that the ability to copy and paste those strings verbatim into code is useful.
- Tom noted an additional use case; strings encoded as Modified UTF-8. Modified UTF-8 requires use of escapes to encode U+0000 as an overlong two-byte sequence.
- Markus added that the same use case applies to creation of CESU-8 strings; escape sequences are needed for the individual encoding of UTF-16 surrogate pairs.
- Tom stated that it is useful to embed a null terminator with `\0`, though it would still be possible to do so using `\u0000`.
- Mark observed that implementations can warn if a literal that contains numeric escape sequences produces an ill-formed UTF string.
- Poll: Continue to allow hex and octal escapes that indicate code unit values, requiring only that they fit into the range of the code unit type.

S F F N A S A

8 1 0 0 0

- **char8_t:**

- Zach started the discussion by noting that use of `char8_t` does not help to enforce preconditions; ill-formed UTF-8 can appear in sequences of `char8_t` just as it can in sequences of `char`. How does `char8_t` help?
- Mark acknowledged that preconditions can always be violated.
- Tom offered `make_text_view` and UDLs as examples. `char8_t` enables writing generic functions that work with ordinary and UTF-8 string literals.
- Zach summarized, I see, it allows authors of overload sets to differentiate behavior.
- Markus chimed in, starting to see the motivation for `char8_t`; generic code can't distinguish encodings unless it is represented in the type system.
- Markus further noted that the standard library has a high percentage of generic code relative to code outside the standard.
- Tom agreed, but noted there is more focus on generic libraries now than in the past and that the committee is working hard to improve support for generic programming as exemplified by Concepts.
- Tom mentioned that we have multiple encodings we have to support.
- Markus acknowledged the dilemma; many other languages have settled on a single internal encoding, but C++ supports multiple encodings and there is no clear dominant one across the industry.
- Mark added that there is considerable baggage with `char` and the implementation definedness of the execution encoding.
- Markus acknowledged the existence of many incompatible string types in C++ that are all similar in intent.
- Tom stated that Concepts helps to bring these different string types together such that they can be supported by generic code.
- Markus observed that the `char8_t` proposal changes existing behavior.
- Mark noted that `u8` literals aren't used much in C++.
- Markus mentioned that Google uses `unsigned char` and ensures use of UTF-8 internally.
- Tom responded that there is a backward compatibility story that is aided by C++20 support for class types as non-type template parameters as proposed in [P0732](#).

- **Code points vs grapheme clusters:**

- Martinho lead the discussion by expressing concern that grapheme cluster boundaries are not stable. The situation with Swift today is that behavior depends on the version of ICU installed on the system. Behavior is therefore non-portable.
- Mark mentioned that we have a similar issue with the timezone database and `<chrono>`. Behavior depends on which version of the database is installed.
- Tom acknowledged the concern; we won't have portable grapheme breaking in C++ either.
- Markus provided a link to a recent document authored by Mark Davis and noted a limitation imposed by the instability of grapheme cluster boundaries; stored EGC indexes are invalidated when changing Unicode versions.
 - https://docs.google.com/document/d/1wuzzMOvKOJw93SWZAqoim1VUI9mloUxE0W6Ki_G23tw/edit
- Zach asked, as someone without a lot of end user experience, how often do programmers make poor choices regarding handling of Unicode text?
- Steve responded that he sees bug reports frequently where programmers inadvertently sliced grapheme clusters.
- Martinho provided links to a couple of example defects:

- <https://bugs.swift.org/browse/SR-3582>
- <https://stackoverflow.com/questions/26862282/swift-countelements-return-incorrect-value-when-count-flag-emoji>
- Tom asked, so how do we make a decision about how to proceed.
- Martinho countered that we don't need to yet.
- Steve chimed in with, how do we make them less scary?
- Mark responded with a question, how are things going to look? New types on top of `std::string_view` and `std::string`?
- Zach provided a brief overview of how Boost.Text handles grapheme clusters.
- Markus asked, does Boost.Text enforce well-formed UTF-8?
- Zach responded that it encourages, but does not require well-formed UTF-8.
- Markus mentioned that validation can be expensive. If you know your input is well-formed, then lookups can be optimized without having to decode.
- Tom described this as a design trade off; validate up front and reap performance benefits later, or skip validation and lazily validate later.
- Markus noted that it is common for programmers to slam content into strings and then validate them later.
- Mark mentioned that [P1072](#) helps to support that use case.
- Tom asked, assuming that we standardize a type that enforces well-formedness, is there room for standardizing a non-validating type as well? Or does that become an expert level do-it-yourself feature?
- JeanHeyd advocated an adapter-over-range approach for `std::text`; tags can suppress validation when it isn't necessary.
- Tom observed that it isn't possible to enforce well-formedness on views without introducing validation costs.
- Steve mentioned that adapters over containers make memory allocation someone else's problem, for better or worse.
- Martinho advocated that, if performing validation on container construction, would prefer replacement character substitution since throwing gives you nothing. Invalid input can be used as an attack vector; if UTF-8 input is all `0x80`, replacement will triple the buffer size.
- Zach expressed openness to an adapter approach for Boost.Text.
- Mark expressed a preference for the adapter approach as it supports underlying containers with reference counts or small buffer optimizations.
- Mark also mentioned that wrapping `std::string` provides a nice transition story.
- Tom then summarized the plan for the San Diego meeting: discussion of the [Unicode Direction paper](#), [P1072](#), Isabella Muerte's [P1275](#), and then small groups to focus on further proposal incubation.

December 5th, 2018

Draft agenda:

- Draft guidelines for other WGs and SGs to request SG16 review.
- `char8_t` remediation for backward compatibility impact.
- Review P1072 following San Diego LEWGI feedback.

Attendees:

- Bryce Adelstein Lelblach
- Cameron Gunnin
- Corentin Jabot
- Florin Trofin
- JeanHeyd Meneide
- Mark Zeren
- Markus Sherer
- Peter Bindels
- Steve Downey
- Tom Honermann
- Zach Laine

Meeting summary:

- Draft guidelines for other WGs and SGs to request SG16 review.
 - Tom introduced the topic. Bryce had suggested that SG16 produce a rubric detailing guidance for when other WGs and SGs should consult SG16. SG7 recently produced such a document. Tom felt this was an excellent idea and is now bringing it before SG16 for discussion.
 - Tom first asked Bryce where SG7's rubric can be found.
 - Bryce replied that it will be in the San Diego post-meeting mailing.
 - Tom then asked for suggested guidance.
 - Steve suggested a simple litmus test; "if it smells like Unicode..."
 - Corentin mentioned having discussed this with Titus in San Diego and suggested that anything having to do with text processing should be sent our way.
 - Bryce asked about locales and it was agreed that Unicode has locale dependencies.
 - Peter mentioned the `{fmt}` library; code units vs code points?
 - Tom replied that we discussed `{fmt}` with Victor in SG16 on several occasions.
 - Bryce asked if `{fmt}` is in C++20 and whether SG16 has any concerns about it.
[Editor's note: not yet, but it passed LEWG review in San Diego].
 - Zach replied that it is certainly no worse than what we have now.
 - Mark commented, bird in hand... even if we had issues with the `{fmt}` library, there is no competing proposal.
 - Corentin mentioned that `{fmt}` does not yet handle `char16_t` and `char32_t`, but can be extended later.
 - JeanHeyd elaborated, template overloads are present, but formatting strings must be `char` or `wchar_t` at the moment.
 - Zach suggested a requirement; that we need to reserve the right to explicitly specialize standard library templates that might be instantiated by users with `char8_t`.
 - Tom asked for a volunteer to identify such templates.
 - Zach volunteered. Hooray for Zach!
 - Steve suggested that anything involving command lines, file names, and environment variables should be sent our way.
 - Mark added, any kind of encoding. Including source encoding.
 - Tom asked, do we want SG13 (HMI) members consulting us for text input and presentation issues?
 - Steve replied, when they get to that point, yes.
 - Tom asked for a volunteer to draft the rubric paper.
 - Steve volunteered. Hooray for Steve!
- `char8_t` remediation for backward compatibility impact.
 - Tom gave a brief introduction and pointed the group at a rough draft paper posted to the mailing list (<http://www.open-std.org/pipermail/unicode/2018-December/000180.html>).
 - Time was given for those who had not yet seen it to quickly scan it.
 - Steve commented on the proposed change to make ostream inserters for `char16_t` and `char32_t` ill-formed; for anyone actually relying on printing pointer values, a fix should be easy, add a cast to `void*`.
 - Corentin wondered if anyone actually does `std::cout << u8"text"`.
 - Zach observed that someone could conceivably want to use the ostream inserters to print `char16_t` values formatted as hex integers, say when dumping UTF-16 code units for diagnostic purposes.
 - Steve asked if it would be problematic to allow `std::string` to be constructed with `char8_t` based data.
 - Zach responded that he didn't see any harm.
 - Peter chimed in that `std::string` always holds UTF-8 in the code base he works on.
 - Tom stated that supporting `std::string` interoperability with `u8` literals would require a lot of overloads for the `char` based specialization of `std::basic_string`. Implementors would not like that.
 - Zach asserted that he wants, somehow, to be able to construct `std::string` objects initialized with `u8` literals.
 - Tom asked if using a factory function would suffice.
 - Zach responded that would require updates and therefore doesn't address existing code.
 - Markus advised thinking of `std::string_view` in addition to `std::string`.
 - JeanHeyd asked about allowing `std::u8string` to be convertible to `std::string`.
 - Tom stated he thought that might allow most existing code to just work. But, would we really want that? Implicit conversions are often undesirable.
 - Peter responded that he thought so, yes. Existing code mixes UTF-8 with `char`.
 - Corentin observed that implicit conversion from `std::u8string` could lead to mojibake.
 - Zach acknowledged that `std::string` doesn't guarantee any encoding.
 - Peter asked about the possibility of making it UB for `std::u8string` to contain non-UTF-8 data.

- Zach requested not adding encoding guarantees for strings.
- Peter responded, it doesn't actually work anyway since you couldn't update a string without introducing UB.
- Tom asked if the UDL approach to providing UTF-8 data in char via u8 literals was realistic.
- Zach stated we shouldn't be suggesting macros as solutions.
[Editor's note, macros are not required to create a solution that works for C++17 and C++20, but source code changes are required].
- Tom asked if use of `-fno-char8_t` is a valid option noting that it forks the language.
- Zach suggested, perhaps this is our first good opportunity to put tooling to use as part of a C++20 migration story.
- Corentin observed that it should be easy to use `clang-tidy` to update code.
- JeanHeyd asked if `char8_t` could implicitly convert to `char`.
- Corentin stated that he wants conversions to be explicit.
- Tom mentioned that the draft paper is intended to tell a migration story.
- Markus explained that he felt the economics are not right. The current situation puts the burden of addressing breakage on many programmers.
- Zach suggested adding tooling automation to the paper.
- Tom said he could add `clang-tidy`, what else should be mentioned?
- Zach stated he'd like to see compilers do fix-ups themselves.
- Corentin observed that implementors are unlikely to have something in place in the necessary time frame.
- Tom asked about experimentation.
- Peter stated his code base isn't using u8 literals today and won't be able to.
- Markus observed that not all code is equally modifiable. For example, Google's code base has a lot of Google specific code, but also uses a lot of third party code. Updating the third party code and potentially maintaining differences from upstream, is more difficult than updating Google's own code.
- Tom suggested a C++17 compatibility library could be made available that implements some of the remediation approaches noted in the draft paper.
- Bryce asked about the possibility that the `char8_t` proposal might be re-litigated due to backward compatibility concerns.
- Tom replied, sure, anything is possible.
- Bryce suggested adding data about expected breakage to the remediation paper to avoid scaring people.
- Peter requested time in SG16 for presenting and collecting feedback on a simple 2D graphics library he has been working on.

December 19th, 2018

Draft agenda:

- Continue discussion of `char8_t` remediation for backward compatibility impact.
 - Discuss pros/cons of keeping u8 literals char based and introducing new `char8_t` based U8 literals.
- Review P1072 following San Diego LEWGI feedback.

Attendees:

- Bryce Adelstein Lelblach
- JeanHeyd Meneide
- Mark Zeren
- Peter Bindels
- Steve Downey
- Tom Honermann

Meeting summary:

- Continued discussion of `char8_t` remediation for backward compatibility impact.
 - Tom introduced the discussion topic. One approach to minimizing backward compatibility impact would be to restore u8 literals being char-based and to introduce a new u8 literal prefix for `char8_t` based UTF-8

literals.

- Mark suggested following up with Google folks to determine if this would address their concerns.
- Tom stated he talked to Chandler following the San Diego vote. Concerns expressed were that the potential backward compatibility impact exceeded the benefits.
- Tom asked for pros and cons for a new u8 literal prefix.
- JeanHeyd was first to note the obvious primary benefit, avoids backward compatibility issues.
- Tom agreed, but added that P0482 does have other minor breakage; the changes to the return types of the `u8string` member functions of `std::filesystem::path`.
- JeanHeyd pointed out that the visual difference between `u8` (lowercase) and `U8` (uppercase) is subtle and bad for readability.
- Bryce agreed and pointed out that MISRA forbids identifiers that look similar.
- Bryce further stated that use of `u` and `U` for `char16_t` and `char32_t` literals was a mistake for the same reason.
- Mark mentioned a pro, this approach preserves investment in any increased use of `u8` literals in code over the next few years before migration to C++20.
- Bryce suggested that compiler warnings could be added to help educate programmers about the change when compiling in pre-C++20 language modes. This still depends on compiler upgrades of course.
- Tom agreed and noted that Clang trunk already issues such a warning when invoked with `-Wc++2a-compat`.
- Mark asked if a cast or similar approach for converting `u8` literals to char-based types doesn't suffice.
- Tom responded that Zach expressed a desire for existing code to continue working at our last meeting.
- Tom asked what adopting an additional literal prefix would mean for messaging. What would we be telling programmers going forward? We could deprecate `u8` literals and promote `U8` going forward.
- JeanHeyd responded that deprecation doesn't really help to move programmers towards use of `char8_t`. He'd prefer to break things, get over the migration hump, and keep a cleaner design.
- Mark asked why the `as_char` approach suggested in the draft paper doesn't suffice.
- JeanHeyd responded that it requires markup, so existing code requires changes.
- Mark pondered, a new prefix does kind of fix everything. It doesn't have to be `U8`, we could use `utf8` or similar.
- JeanHeyd suggested we could introduce new prefixes for all of UTF-8, UTF-16, and UTF-32 in order to maintain symmetry and to address the subtle `u` vs `U` concerns.
- Tom suggested another pro; a new prefix avoids potentially forking the language by unintentionally encouraging use of a `-fno-char8_t` option as has happened with `-fno-rtti` and `-fno-exceptions`.
- Mark asked where we're at with proposing `char8_t` to WG14.
- Tom responded that he would like to get a proposal in front of WG14 at their October 2019 meeting in Ithaca. In addition, he'd like to have proposals ready for our other proposals targeting core language features:
 - [P1097](#) - "Named character escapes"
 - [P1041](#) - "Make `char16_t`/`char32_t` string literals be UTF-16/32"
 - Source file encoding tags (no proposal yet).
- Tom added another pro, or con, depending on perspective; a new prefix maintains the ability to continue writing UTF-8 based applications with char-based types.
- Mark opined that moving away from char aliasing issues is compelling.
- Steve noted that UTF-8 in char-based types often seems to work, but works for the wrong reasons. For example, UTF-8 encoded source files compiled as "8-bit ASCII" such that the UTF-8 code units just get copied from the source file.
- Tom asked about messaging again, what message are we sending to library authors? Do they write their UTF-8 based interfaces against `char` or `char8_t`? How do they choose?
- Mark observed that this isn't a new problem. Library authors code against `std::string` today and it isn't a universal string type or a great type for Unicode. We'll have similar concerns with the introduction of `std::text` vs `std::string`.
- Tom concluded, sounds like templates will be the way to go.
- JeanHeyd commented that views help. For example, `text_view` can effectively type erase the code unit type. But what does one assume for encoding for `char`?
- Tom responded that the execution encoding must be assumed per existing precedent in the standard.
- Mark concluded that he doesn't see a way out of the `char` vs `char8_t` problem. But, with `char8_t` being available, we'll get experience using it that will inform future library efforts. In the short term, being able to use either `char` or `char8_t` is advantageous.
- Peter chimed in from chat (due to a non-functioning microphone):

- "looks like my mic is completely broken. From what I can tell this is like the uptake of `uint8_t`, it takes some time but over time everybody learns that these types have a given fixed meaning and others are a :shrug: type"
- Tom presented a few polls.
 - Poll 1: Add defined-as-deleted overloads for `operator<<` for `basic_ostream<char, ...>` specializations.
SF F N A SA
 3 3 0 0 0
 - Poll 2: Allow deprecated `std::filesystem::u8path` to be called with sources with `char8_t` value type.
SF F N A SA
 2 3 0 1 0
 - Peter explained his against vote; this maintains working around something that we don't really want to work in the first place.
 - Poll 3: Restore char-based u8 literals and introduce new `char8_t` based literals with a new prefix.
SF F N A SA
 1 3 1 1 0
 - Bryce explained his against vote; we'll need to converge on a very short prefix, 2 characters at most. That seems unlikely.
 - JeanHeyd commented that he still prefers to go with a solution that pushes the community in a new and consistent direction. u8 literals aren't widely used, so we still have time to course correct.
 - Mark asked if tooling could be used to fix existing code by converting u8 literals to ordinary literals encoded with escapes.
 - Tom responded that we discussed tooling possibilities at the last meeting. Specifically Zach's suggestion that this could be a good test for Titus' goals for tooling.
 - Poll 4: Assuming u8 literals remain `char8_t` based, allow char arrays to be initialized with u8 string literals.
 - Tom stated that the reason to consider this is that the `as_char` approach doesn't work for array initialization.
 - Bryce stated he wanted more time to think about this.
 - Mark agreed with wanting more time.
 - Poll not taken.
- Review P1072 following San Diego LEWGI feedback.
 - Mark provided a summary of changes:
 - No buffer moving features; feedback from San Diego was negative regarding that due to exposure of implementation details.
 - `resize_default_init()` resizes the string such that the added content is default initialized. Failure to write to the added elements results in undefined behavior.
 - This approach matches Google's existing implementation.
 - This approach is compatible with existing allocators.
 - `libc++` is already using this approach as part of its `std::filesystem` implementation to remove an allocation.
 - This doesn't preclude a buffer migration feature in the future.
 - The paper establishes that `basic_string` is allocator aware.

January 9th, 2019

Draft agenda:

- Preparation for the Kona pre-meeting mailing deadline on 1/21.
 - Review the SG16 rupric assuming a draft is available.
 - Review the `char8_t` remediation paper assuming a revision is available.
 - Review other papers requiring an update for Kona (P1041, P1097).

Attendees:

- Cameron Gunnin
- JeanHeyd Meneide
- Mark Zeren
- Michael Spencer
- Steve Downey
- Tom Honermann
- Victor Zverovich
- Zach Laine

Meeting summary:

- Tom stated that he was unable to get a revision of the `char8_t` remediation paper ready for this meeting, so no further discussion on it for now.
- We then started reviewing Steve's [draft SG16 rubric](#).
 - Victor asked about locales as he and Howard have been working on chrono updates that add overloads based on locale.
 - Tom said, yes, bring to SG16 anything involving locales.
 - Zach expressed a preference for just those locale features that relate to Unicode.
 - Tom stated a preference for having a chance to offer our expertise; to help ensure appropriate use of locales.
 - Michael asserted that we don't want new Unicode stuff dependent on `std::locale`.
 - Zach observed that it is very hard to write portable code that uses `std::locale` due to implementation defined things. For example,
 - the set of locales is not specified.
 - even the "C" locale is not portable.
 - Tom suggested that the language regarding "requires review" by SG16 be softened as we don't have standing to actually require review.
 - Zach disagreed and offered the perspective that this paper should be adopted by the LEWG and EWG chairs with the expectation that the chairs will enforce review requirements.
 - Tom expressed enthusiasm for that perspective; this paper should be targeted to LEWG and EWG to get their buy-in.
 - Tom asked about the SG-7 rubric in the hopes that we could compare/contrast with it.
 - Michael located it and provided a link:
 - http://wiki.edg.com/pub/Wg21sandiego2018/SG7/d1354r0.html?twiki_redirect_cache=c261eae64220cb36ab24bdb6fb29d4c
 - Tom suggested we should have a section on text containers and string builders.
 - Zach asked if we care about string builders. If a string builder is used in such a way that it slices code unit sequences, isn't that just an incorrect use of the builder?
 - Tom stated he wants to catch any new operations that are problematic for some encodings. For example, reliance on broken interfaces like `std::ctype::widen`
 - Cameron suggested we're interested in any new overloads involving Unicode types.
 - Zach proposed adding a section detailing encoding assumptions.
 - Tom agreed and suggested that can appear in the text encoding section; we need to make it explicit that char based values of unknown origin are assumed to have execution encoding.
 - Zach disagreed with the assumption of execution encoding stating that they should instead have an unknown encoding and their contents should only be forwarded and operated on generically (e.g., as a bag of bytes), not examined as having data in any particular encoding.
 - Tom challenged this noting that reasonable assumptions can be made. On Windows, execution encoding matches the system code page, on POSIX it corresponds to the `LANG` or `LC_CTYPE` environment variables, and is generally ASCII elsewhere (except z/OS).
 - Zach noted that assumption doesn't work for file names.
 - Tom agreed that filenames are special; they don't have a known encoding. But C++17 at least offers `std::filesystem` with means to get a filename in a displayable format via the `*string` and `generic_*string` member functions of `std::filesystem::path`.
 - Zach asserted those member functions are a trap; the names retrieved via those member functions don't necessarily round trip.

- Michael observed that programmers need to be able to display file names and, if the standard doesn't provide a way to do it, programmers will do it themselves, probably badly.
- Steve noted that file names may not be presentable at all.
- Michael reiterated that we need interfaces that do the right thing easily; e.g., to create a display name for a file in something other than `std::filesystem::path`.
- JeanHeyd observed that some of these problems would go away with a new I/O layer that uses `std::filesystem::path` instead of `const char*` interfaces.
- Steve noted that we can't replace the OS interfaces though.
- Tom stated that we need to update the paper to require consultation with SG16 for anything involving file names.
- [P1378R0: std::string_literal](#)
 - JeanHeyd provided a link to an updated draft revision of the paper:
 - https://the-phd.github.io/vendor/future_cxx/papers/d1378.html
 - JeanHeyd introduced the motivation; to provide means to guarantee that a string literal is used in invocations of `std::embed` in order to enable dependency discovery in build systems. Additional motivation is to provide means to avoid unintended array-to-pointer decay and to handle string literals with embedded null characters without having to depend on deduction via array reference in order to obtain the actual array size of the literal.
 - JeanHeyd acknowledged that the proposal changes the type of all string literals in ways that are unlikely to be acceptable.
 - Michael observed that the proposed design doesn't actually meet the motivation requirements for `std::embed` since the proposed type is copyable and therefore can be produced by many kinds of expressions, not just literals.
 - Steve suggested another motivation: requiring string literals for things like format strings and SQL; requiring a literal would avoid the possibility of consuming user provided input that could be used as an attack vector as in SQL injection attacks.
 - Zach observed that immediate (`constexpr`) functions can help in this regard since they can't consume run-time input by design.
 - Tom asked about a different implementation strategy; making all of the class constructors private and befriending a UDL. This would ensure the class could only be constructed by calling a UDL (assuming copy constructors are deleted).
 - Michael suggested the constructors could also use compiler magic to require construction via a literal.
 - Steve noted that having the size of a string literal readily available would be useful.
 - Michael noted that this design impacts type deduction for auto declared variables and template parameters.
 - Zach suggested that two-step conversion as would be required for backward compatibility would be problematic.
 - JeanHeyd responded that any number of builtin implicit conversions are already permitted.
 - Tom wondered if the number of conversions might impact overload resolution.
 - JeanHeyd suggested the design might be useful to limit when error handling and encoding validation would be necessary for `std::text`.
 - Zach countered that string literals can form ill-formed code unit sequences.
 - Zach acknowledged that the ability to avoid `strlen` could be a big deal.
 - Michael asserted that the motivational use cases can largely be met with immediate (`constexpr`) functions.
 - JeanHeyd provided an additional motivation; comparison between string literals. Today, whether `"foo" == "foo"` is unspecified. The proposed `std::string_literal` could make such comparisons work as expected.
 - Mark asserted that an implementation is needed to evaluate backward compatibility impact.
 - Mark noted having previously had a desire to determine if a pointer pointed to a string literal; to avoid storing the string contents.
 - Zach and Tom both expressed having used or encountered string pool classes that exist to collapse matching strings to a single copy.
- WG21 Direction group [response to P1238R0: SG16: Unicode Direction](#)
 - Steve summarized the response.
 - Tom noted that the DG did not comment on the constraints listed in the paper.
 - Mark noted the DG request to clarify scope.
 - Zach stated that we need an elevator pitch and suggested: We want all Unicode algorithms available via standard interfaces for C++23.
- Tom announced that the next meeting will start an hour later than usual.